

Here is the contextual background and breakdown for each slide of the presentation:

Slide 1: Title Slide (Cover)

- **Context:** Sets the stage for the project presentation delivered by Hope Donglo, a Freelance Data Engineer, in March 2026.
 - **What is done:** Presents the project identity as a Proof of Concept (PoC) for **Puls-Events**, a cultural assistant in Paris that uses Retrieval-Augmented Generation (RAG) to recommend events.
-

Slide 2: Context and Objectives

- **Context:** Generic Large Language Models (LLMs) often hallucinate or lack access to up-to-date, hyper-local event data, which makes them unreliable for real-time cultural recommendations.
 - **What is done:** Identifies the core problem with Puls-Events' aggregation needs and introduces the **RAG Architecture Solution**—combining vector-based retrieval with grounded French generation to ensure zero hallucinations and current event data.
-

Slide 3: System Architecture

- **Context:** A clear division between the offline data processing pipeline and the online user interaction flow is required for system efficiency.
 - **What is done:** Maps out two primary workflows:
 1. **Batch Pipeline:** Raw Open Agenda data (9,651 events) → Cleaning (1,740 retained) → Embedding (**mistral-embed**) → FAISS indexing (1,024 dimensions) → Mistral LLM.
 2. **Real-time Flow:** A 6-step sequence starting from the user prompt, converting it to an embedding, running a top-5 FAISS search, injecting context, generating the answer with Mistral, and displaying sources.
-

Slide 4: Data Source (Open Agenda Paris)

- **Context:** Data quality and freshness are critical for preventing hallucinations and providing valid recommendations.

- **What is done:**
 - Explains why native API v2 (Agenda UID 82290100) was selected over outdated OpenDataSoft exports (which contained static 2021 data).
 - Details data cleaning rules: filtered down to 1,740 relevant events within a 12-month window across 97 paginated API calls.
 - Highlights data fixes: removed `keywords.fr` (89% missing values), validated `description.fr` (0% missing), normalized Unix timestamps, and corrected postal codes in the location field.
-

Slide 5: Technical Choices

- **Context:** Justifies the technology stack decisions made for performance, speed, cost, and reproducibility.
 - **What is done:**
 - **mistral-embed:** Selected for native French text support and vendor consistency with the generation model.
 - **FAISS IndexFlatL2:** Chosen for exact vector similarity searches under 1 ms across 1,740 vectors.
 - **mistral-small-latest:** Configured with low temperature (0.1) and strict system prompts to prevent hallucination.
 - **Environment & Tooling:** Poetry with Python 3.13 for reproducible dependency locking.
 - **Chunking Strategy:** Single chunk per event concatenating title, description, venue, and date.
 - **UI:** Built using Streamlit with resource caching (`@st.cache_resource`).
-

Slide 6: Why LangChain? (Pipeline Orchestration)

- **Context:** Explains the design decisions behind using an orchestration framework rather than direct API calls.
- **What is done:**
- Explains that LangChain abstracts LLM and vector database providers, allowing easy model switching (e.g., to GPT-4 or Llama) without refactoring vector search logic.

- Highlights the single LCEL pipeline (`src/rag/langchain_chain.py`) shared across both the live Streamlit chatbot and the evaluation script, ensuring real-world feature parity during testing.
-

Slide 7: Automated Pipeline

- **Context:** Automating ETL and verification steps prevents corrupted indexes from making it to production.
 - **What is done:** Illustrates an automated workflow that fetches, chunks, embeds, indexes, and validates data across 24 quality checks. If validation fails, the pipeline halts; if successful, it proceeds to evaluation and live deployment. Smart caching skips completed steps unless `--force` is used.
-

Slide 8: Results – Retrieval Quality

- **Context:** Validates how well the vector database retrieves correct matches and handles off-topic or out-of-scope prompts.
 - **What is done:** Demonstrates distance thresholds ($L_2 < 0.50$ considered relevant; $L_2 > 0.60$ treated as out-of-scope). Shows success on event, location, and date queries, as well as a graceful failure response (“Je ne sais pas”) when asked for restaurant recommendations.
-

Slide 9: Live Demonstration

- **Context:** Provides a visual mockup of the final user experience within the Streamlit chat application.
 - **What is done:** Displays a sample prompt (“Quels concerts classiques ce mois ?”) alongside structured recommendations and clickable source citations.
-

Slide 10: Unit Testing (25 / 25 Passed)

- **Context:** Verifies system stability and data integrity prior to running evaluation routines or deploying.
- **What is done:** Shows full test execution passing 25 pytest checks:
- 6 tests for raw data presence and non-null values.
- 3 tests for valid date ranges (within 12 months).

- 3 tests for location filtering (over 90% Paris coverage).
 - 10 tests for vector DB health (1,024 dimensions, float32, sync checks).
 - 3 tests for chunk content formatting.
-

Slide 11: Evaluation & Test Dataset

- **Context:** RAG performance must be systematically measured using standard frameworks rather than manual testing.
 - **What is done:** Evaluates an annotated dataset of 30 test questions across 6 query categories (specific events, dates, categories, locations, out-of-scope, no matches) using RAGAS metrics (**faithfulness**, **answer_relevancy**, **context_precision**, **context_recall**).
-

Slide 12: PoC Limitations & Production Recommendations

- **Context:** Outlines what needs to evolve between a functional PoC and an enterprise-grade production environment.
 - **What is done:**
 - **Identifies Limitations:** Single data source, manual pipeline execution, free-tier rate limits, and un-persisted FAISS indexing.
 - **Proposes Solutions:** Scheduled orchestration (Airflow/Kestra), managed vector databases (Qdrant/Pinecone), paid API tiers, multi-source aggregation, incremental indexing, and active monitoring.
-

Slide 13: Summary & Next Steps

- **Context:** Concludes the project findings and proposes final actions for deployment.
 - **What is done:** Summarizes project milestones: verified zero-hallucination RAG architecture, 1,740 Paris events indexed, 24 unit tests passed, and 30 annotated evaluation questions processed. Recommends moving forward with production planning.
-

Slide 14: Questions / Contact

- **Context:** Closing contact slide for project handoff and Q&A.

- **What is done:** Provides author contact information, GitHub repository pointers, and dataset location paths.
-

Slide 15: Submitted Deliverables

- **Context:** Final checklist validating compliance with platform submission standards.
- **What is done:** Lists all 7 required project deliverables (README, Dependency Lockfile, Processing Scripts, Unit Tests, Technical Report, RAG Source Code, and Presentation) along with mandatory platform file-naming conventions.

This script is the **data ingestion phase** of a Retrieval-Augmented Generation (RAG) pipeline focused on cultural events in Paris.

It acts as an automated pipeline step: it sets up a Python script, executes it to fetch live event data from the Open Agenda API, cleans and structures that data, and saves it locally for downstream embedding and retrieval.

Here is a structured breakdown you can use during your defense:

1. High-Level Overview

- **Goal:** Extract real-world cultural event data from the **Open Agenda API**, filter it for relevance, format it into clean JSON, and record a timestamp metadata file.
 - **Architecture Role:** Step 1 of a RAG system. Before you can generate embeddings (`02_embed.sh`) or answer queries, you need a fresh, clean dataset stored locally (`data/raw/events_paris.json`).
-

2. Key Components & How They Work

A. The Shell Script (`01_fetch.sh`)

- **Environment Checks:** Verifies that a `.env` file (containing API credentials) and a virtual environment (`.venv`) exist before running.
 - **Self-Contained Setup:** Dynamically generates `src/ingestion/fetch_events.py` using a bash `cat << 'EOF'` HEREDOC block, ensuring all folder structures and code exist in the right place.
 - **Execution & Force Flag:** Executes the Python script inside the virtual environment. Accepts an optional `--force` argument to override caching and re-download data.
-

B. The Python Ingestion Script (`fetch_events.py`)

1. **Config & Environment Loading (`load_config`):** Reads the Open Agenda API key, agenda UID (defaulting to Paris events), and retention limit (`MAX_EVENT_AGE_DAYS`, default 365 days) from the `.env` file.
2. **Resilient Network API Fetching (`build_session` & `fetch_page_with_retry`):**
 - Uses **cursor-based pagination** to step through thousands of events 100 at a time.
 - **Dual Layer Resilience:**
 - *Layer 1:* `urllib3` retry adapter handles standard HTTP status errors (e.g., 429 Too Many Requests, 500/502/503/504).
 - *Layer 2:* A custom loop with **exponential backoff** (waiting 2s, 4s, 8s, 16s, 32s) handles TCP connection resets, chunked encoding errors, or timeouts mid-stream.
3. **Data Filtering & Cleaning (`pd.json_normalize`, `filter_by_date`, `clean_nulls`):**
 - Flattening nested JSON structures into a Pandas DataFrame.
 - Dropping events older than 1 year or further than 12 months in the future.
 - Trimming columns down to essential fields (`uid`, `title`, `description`, `city`, `address`, `dates`, etc.).
 - Removing rows missing vital context (null titles or descriptions) and standardizing city names (e.g., handling online events as "En ligne" and fixing "75015 Paris" to "Paris").
4. **Validation & Metadata Tracking (`validate`, `save_fetch_metadata`):**
 - Runs strict assertion checks to ensure empty or broken data is never passed downstream.
 - Writes `fetch_metadata.json` recording `fetched_at` (UTC timestamp). This allows downstream unit tests to evaluate relative dates consistently, preventing tests from breaking over time.

3. Defense Talking Points (Quick Bullet Points)

If asked about design choices during your presentation:

- **Why exponential backoff?** *"Open Agenda's API drops long-lived TCP connections after many pages. Manual retry with exponential backoff prevents the entire script from failing midway through a large dataset."*
- **Why store `fetch_metadata.json`?** *"RAG systems rely on time-sensitive context. Storing the exact fetch timestamp ensures our unit tests validate relative dates (e.g., 'is this event in the past?') against when the snapshot was taken, making tests deterministic."*

- **Why filter fields early?** *“We prune unnecessary API fields before saving raw data to reduce disk usage and optimize vector chunking/embedding in Step 2.”*

This script represents **Step 2** of your RAG pipeline: **Document Processing & Vector Embedding**.

It takes the raw event data fetched in Step 1, transforms it into optimized text chunks containing both semantic content and metadata, and uses the **Mistral API** (`mistral-embed`) to convert those text chunks into dense mathematical vector representations.

Here is a structured, defense-ready breakdown of what this script does and why it was built this way.

1. High-Level Pipeline Overview

- **Input:** Raw JSON data from Step 1 (`data/raw/events_paris.json`).
- **Process:**
 1. Concatenates key fields (Title, Description, Venue, City, Dates) into single structured text units (1 chunk per event).
 2. Generates semantic embeddings for each chunk via Mistral AI in rate-limited batches.
- **Output:**
 - `data/processed/chunks.json`: Standardized text chunks paired with downstream metadata (URLs, dates, venues).
 - `data/processed/embeddings.npy`: A 2D NumPy array matrix storing 1024-dimensional floating-point vectors.

2. Deep Dive: Key Components & Technical Choices

A. The Chunker Strategy (`chunker.py`)

- **1 Event = 1 Chunk:** Traditional RAG systems often split long documents using recursive text splitters (e.g., 500-token sliding windows). Here, the code explicitly uses a 1:1 mapping.
- **Why?** Event descriptions in Open Agenda are very short (median ~50 characters). Splitting them would lose critical context.
- **Context Enriched Text (`build_chunk_text`):** Instead of embedding *only* the raw description, it builds a formatted block combining:

```
Événement : <title>
Description : <description>
```

Lieu : <venue>, <city>
Date : <date>

- **Why?** Adding explicit labels (“Lieu:”, “Date:”) helps the embedding model capture location-based and date-based semantics along with the descriptive text.
 - **Metadata Attachment:** Attaches the Open Agenda URL, venue name, address, and temporal parameters (`date_begin`, `date_end`) directly to the chunk dictionary so the generation/retrieval engine can cite sources and display metadata to the user later.
-

B. The Embedder Strategy (`embedder.py`)

- **Model:** Uses `mistral-embed` (producing **1024-dimensional dense vectors**).
 - **API Rate-Limit Resilience:**
 - **Small Batching:** Chunks are sent in small batches of **5** (`BATCH_SIZE = 5`) with a forced 3-second sleep (`SLEEP_BETWEEN = 3.0`) between calls.
 - **HTTP 429 Backoff:** If the Mistral API returns a Rate Limit error (429), the script intercepts the exception, sleeps for **60 seconds**, and re-attempts the batch once before raising a hard failure.
 - **Storage:** Saves vectors directly as a binary NumPy file (`.npy`). This provides fast disk read/write access and minimal memory overhead when loading data into a vector index in Step 3.
-

C. Sanity Checks & Assertions

Before saving files, the script enforces two critical defensive checks:

1. `assert len(chunks) == len(events)` — Confirms no events were accidentally dropped during chunking.
 2. `assert embeddings.shape[1] == 1024` — Guarantees vector dimensionality strictly aligns with the `mistral-embed` spec before downstream indexing.
-

3. Key Defense Talking Points & Anticipated Questions

If your examiners ask about design choices in Step 2:

- **“Why didn’t you use a recursive text splitter (e.g., LangChain’s RecursiveCharacterTextSplitter)?”** > *“Event items are self-contained, micro-documents. Applying standard fixed-size chunking would fragment the venue from its description or break date context. Formulating 1 enriched chunk per event preserves the complete semantic context of each event in a single vector.”*
- **“Why store embeddings in a .npy file instead of directly in a database?”** > *“Decoupling embedding generation from index construction is a best practice. Generating embeddings incurs API cost and time. Storing raw vectors as a .npy file creates a lightweight persistence layer so we can experiment with different vector databases (e.g., FAISS, Chroma, Qdrant) in Step 3 without re-paying for API calls.”*
- **“How do you handle API limits during large bulk embeds?”** > *“We implement rate-budgeting directly in the script using small batch sizes (5 items/batch), polite delays (3s), and automated fallback logic that catches HTTP 429 status codes and pauses execution for 60 seconds.”*

This script is **Step 3** of your RAG pipeline: **Vector Store Indexing**.

It takes the 1024-dimensional embeddings (`embeddings.npy`) generated in Step 2 and loads them into a fast vector search engine using **FAISS** (Facebook AI Similarity Search), while maintaining a parallel metadata file (`metadata.json`) to store human-readable fields.

Here is a structured breakdown you can reference during your defense:

1. High-Level Pipeline Overview

- **Input:**
 - `data/processed/embeddings.npy` (1024-D floating-point vectors)
 - `data/processed/chunks.json` (Text chunks and event metadata)
 - **Process:** Builds a ****FAISS IndexFlatL2**** vector index and creates a position-aligned metadata mapping.
 - **Output:**
 - `data/processed/faiss_index.idx` (Binary vector index optimized for fast similarity search)
 - `data/processed/metadata.json` (JSON list where array index i corresponds directly to FAISS vector ID i)
-

2. Key Technical Decisions & Architecture

A. FAISS Index Choice (`IndexFlatL2`)

- **Algorithm:** `IndexFlatL2` computes the exact **Euclidean distance** (L_2 **norm**) between a query vector and every vector in the index (brute-force k-NN).
 - **Trade-off:**
 - **Pros:** 100% recall accuracy (no approximate nearest neighbor loss/compression), lightweight setup, zero hyperparameter tuning required.
 - **Cons:** $O(N)$ query scaling complexity.
 - **Why this makes sense for this project:** For small-to-medium datasets (thousands to tens of thousands of events), `IndexFlatL2` executes in single-digit milliseconds. There is no need for approximate indexing structures (like `IndexIVFFlat` or `HNSW`) until scaling past ~100k vectors.
-

B. Positional Coupling Architecture

FAISS natively indexes vectors by integer positional IDs ($0, 1, 2, \dots, N - 1$) but does **not** store non-vector attributes (like text strings, dates, or URLs).

To solve this, the script creates a **parallel array** (`metadata.json`):

- FAISS vector at position 0 $\rightarrow$ Metadata dict at index 0
- FAISS vector at position k $\rightarrow$ Metadata dict at index k

When a query is run downstream:

1. FAISS returns top- K integer indices (e.g., `[42, 108, 12]`).
 2. The search engine retrieves `metadata[42]`, `metadata[108]`, and `metadata[12]` in $O(1)$ constant time.
-

C. Integrity Validation & Persistence

- **Artifact Check (`load_artifacts`):** Asserts that `len(chunks) == embeddings.shape[0]` and verifies the 1024-dimension vector width before index construction.
 - **Round-Trip Reload Verification (`verify_reload`):** Immediately re-reads the serialized `.idx` file back into memory to confirm `reloaded.ntotal == expected_count`, guaranteeing disk serialization didn't corrupt the file.
-

3. Defense Talking Points & Anticipated Questions

If your committee asks about Step 3:

- “**Why did you choose L_2 (Euclidean) distance instead of Cosine Similarity?**” > “When vectors are normalized, L_2 distance and Cosine Similarity yield identical ranking order. If embeddings aren’t explicitly unit-normalized, L_2 measures absolute distance in the embedding space. *IndexFlatL2* is the standard baseline choice for FAISS prototypes before moving to inner product (*IndexFlatIP*) or HNSW graphs.”
- “**How does your index handle non-vector filtering (e.g., filter events by date or city)?**” > “FAISS manages geometric similarity search. Non-vector metadata (dates, locations, URLs) is decoupled and preserved in a parallel array (*metadata.json*). Downstream retrieval can perform **post-filtering** on the retrieved top- K results using this metadata.”
- “**Will IndexFlatL2 scale if Paris events grow to 1 million?**” > “No, $O(N)$ exact search becomes inefficient around 100k+ vectors. At that scale, we would swap *IndexFlatL2* for an approximate nearest neighbor (ANN) index like **HNSW** (Hierarchical Navigable Small World) or **IVF-PQ** (Inverted File with Product Quantization) to trade a fraction of recall accuracy for logarithmic search speed.”

This script represents the final presentation layer of your system: **The Interactive Web Interface (chatbot.py) built with Streamlit.**

It wires the underlying vector retrieval engine and LLM chain directly to a user-facing chat application, enabling real-time, interactive RAG for Parisian cultural events.

Here is a structured, defense-ready breakdown of what this script does and why its architectural design is notable.

1. High-Level Pipeline Overview

- **Input:** Natural language user queries submitted through a browser chat interface (e.g., “*Concerts ce weekend à Paris*”).
 - **Processing:** Executes the canonical LangChain retrieval-augmented chain (`ask()` imported from `src.rag.langchain_chain`).
 - **Output:**
 - Interactive, conversational response in the main chat canvas.
 - Dynamically updated source references (title, date, venue, direct URL) rendered inside a dedicated sidebar context panel.
-

2. Key Architectural Decisions & Engineering Best Practices

A. Modular Design (No Duplicate Logic)

- **Single Source of Truth:** A common flaw in RAG student projects is writing two separate RAG pipelines: one for automated evaluation/benchmarking and a separate, hand-rolled loop inside the UI script.
 - **The Solution:** This script imports `ask()` directly from `src/rag/langchain_chain.py`. The Streamlit app, command-line scripts, and test suites run on **the exact same LangChain execution path**.
-

B. Dynamic Python Path Resolution

```
ROOT = Path(__file__).resolve().parents[2]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
```

- **Why this matters:** Streamlit changes the working directory depending on where the user executes the `streamlit run` command from. Dynamically appending the project root to `sys.path` guarantees that internal package imports (e.g., `from src.rag.langchain_chain import ask`) succeed seamlessly regardless of execution context.
-

C. UI UX & Grounded Retrieval Transparency

- **Session State Management (`st.session_state`):** Retains the full conversational message history so user interactions are rendered fluidly across Streamlit's reactive re-runs.
 - **Source Context Sidebar (`st.sidebar`):** RAG systems must justify their answers to prevent hallucination concerns. When a response finishes generating, the sidebar populates expanding accordion cards containing:
 - Event Name & Venue
 - Date Ranges
 - Direct clickable URLs back to the source agenda
-

3. Defense Talking Points & Anticipated Questions

If your defense committee asks about the application layer:

- **“Why did you choose Streamlit over a traditional frontend (e.g., React/FastAPI)?”** > *“Streamlit allows rapid prototyping of data and AI applications without decoupling the Python backend from the UI layer.*

For demonstrating and testing a RAG system in an academic or production proof-of-concept setting, Streamlit provides native chat components (`st.chat_message`, `st.chat_input`) and reactivity out of the box.”

- **“How do you ensure the UI demo matches the evaluation results in your report?”** > *“We enforce strict architectural parity. The UI does not contain custom retrieval or LLM call logic; it imports the exact `ask()` function from `src/rag/langchain_chain.py` used by our evaluation scripts. Every prompt and document retrieved in the live demo follows the identical code path measured in our report.”*
- **“How does the app handle source transparency to prevent AI hallucinations?”** > *“The interface explicitly splits generative synthesis from source evidence. The main window displays the LLM’s response, while the sidebar uses Streamlit’s `st.empty()` container to render structured source cards (venue, date range, link) corresponding to the exact vector chunks retrieved from FAISS.”*

This script represents the final presentation layer of your system: **The Interactive Web Interface (`chatbot.py`) built with Streamlit.**

It wires the underlying vector retrieval engine and LLM chain directly to a user-facing chat application, enabling real-time, interactive RAG for Parisian cultural events.

1. High-Level Pipeline Overview

- **Input:** Natural language user queries submitted through a browser chat interface (e.g., *“Concerts ce weekend à Paris”*).
- **Processing:** Executes the canonical LangChain retrieval-augmented chain (`ask()` imported from `src.rag.langchain_chain`).
- **Output:**
 - Interactive, conversational response in the main chat canvas.
 - Dynamically updated source references (title, date, venue, direct URL) rendered inside a dedicated sidebar context panel.

2. Key Architectural Decisions & Engineering Best Practices

A. Modular Design (No Duplicate Logic)

- **Single Source of Truth:** A common flaw in RAG student projects is writing two separate RAG pipelines: one for automated evaluation/benchmarking and a separate, hand-rolled loop inside the UI script.

- **The Solution:** This script imports `ask()` directly from `src/rag/langchain_chain.py`. The Streamlit app, command-line scripts, and test suites run on **the exact same LangChain execution path**.
-

B. Dynamic Python Path Resolution

```
ROOT = Path(__file__).resolve().parents[2]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
```

- **Why this matters:** Streamlit changes the working directory depending on where the user executes the `streamlit run` command from. Dynamically appending the project root to `sys.path` guarantees that internal package imports (e.g., from `src.rag.langchain_chain import ask`) succeed seamlessly regardless of execution context.
-

C. UI/UX & Grounded Retrieval Transparency

- **Session State Management (`st.session_state`):** Retains the full conversational message history so user interactions are rendered fluidly across Streamlit’s reactive re-runs.
 - **Source Context Sidebar (`st.sidebar`):** RAG systems must justify their answers to prevent hallucination concerns. When a response finishes generating, the sidebar populates expanding accordion cards containing:
 - Event Name & Venue
 - Date Ranges
 - Direct clickable URLs back to the source agenda
-

3. Defense Talking Points & Anticipated Questions

If your defense committee asks about the application layer:

- **“Why did you choose Streamlit over a traditional frontend (e.g., React/FastAPI)?”** > *“Streamlit allows rapid prototyping of data and AI applications without decoupling the Python backend from the UI layer. For demonstrating and testing a RAG system in an academic or production proof-of-concept setting, Streamlit provides native chat components (`st.chat_message`, `st.chat_input`) and reactivity out of the box.”*
- **“How do you ensure the UI demo matches the evaluation results in your report?”** > *“We enforce strict architectural parity. The UI does not contain custom retrieval or LLM call logic; it imports the exact*

ask() function from `src/rag/langchain_chain.py` used by our evaluation scripts. Every prompt and document retrieved in the live demo follows the identical code path measured in our report.”

- **“How does the app handle source transparency to prevent AI hallucinations?”** > “The interface explicitly splits generative synthesis from source evidence. The main window displays the LLM’s response, while the sidebar uses Streamlit’s `st.empty()` container to render structured source cards (venue, date range, link) corresponding to the exact vector chunks retrieved from FAISS.”

This script represents the **Automated Testing & Quality Assurance Suite** (`test_data_pipeline.py` via `pytest`) for your RAG pipeline.

It validates data integrity, business requirements (temporal and spatial constraints), and vector database synchronization to ensure the RAG system strictly operates on clean, compliant Parisian event data.

Here is a structured, defense-ready breakdown of what this script does and why it was constructed this way.

1. High-Level Pipeline Overview

- **Input:** Raw JSON data, fetch metadata, processed text chunks, `.npy` vector arrays, FAISS `.idx` indices, and `metadata.json`.
- **Process:** Executes 5 test groups covering file existence, field completeness, date filters, geographical constraints, chunk formats, and vector store alignment.
- **Output:** Standardized PyTest summary report asserting pipeline validity (100% pass required before system deployment).

2. Key Test Groups & Architectural Highlights

A. Group 1 & Group 5 — Data Completeness & Chunk Quality

- **Fields Assertions:** Asserts that no events contain null or empty titles (`title.fr`), descriptions (`description.fr`), or start dates (`firstTiming.begin`).
- **Chunk Marker Verification:** Ensures every text chunk begins with the structured prefix “`Événement :`” and retains valid Open Agenda URLs (`url`), guaranteeing downstream vector searches return rich, clickable source references.

B. Group 2 — Temporal Recency Verification (Deterministic Reference Testing)

- **The Problem:** RAG systems that filter by date (e.g., “events less than 1 year old”) can break over time if tests evaluate against `datetime.now()`. Running tests 6 months after initial fetch would falsely fail valid historical snapshot data.
 - **The Solution (`fetch_reference_time`):** Tests check event dates ***relative to `fetch_metadata.json`*** (`fetch_at` timestamp).
 - **Why this is critical:** It decouples *data staleness* (which is refreshed by re-running `01_fetch.sh --force`) from *filter logic correctness*.
-

C. Group 3 — Geographical Boundary Enforcement

- **90%+ Paris Coverage Rule (`test_majority_events_in_paris`):** Verifies that at least 90% of ingested records belong specifically to Paris.
 - **Postal Code Normalization (`test_no_postal_code_anomalies`):** Ensures cleaning logic stripped raw postal codes (e.g., converting “75015 Paris” → “Paris”) to maintain consistent location metadata for vector searches.
-

D. Group 4 — Vector Database Alignment & Synchronization

- **Count Synchronization:** Asserts $N_{\text{vectors}} == N_{\text{chunks}} == N_{\text{metadata}}$ (1 : 1 : 1 positional sync between FAISS vectors, text chunks, and metadata dictionaries).
 - **Dimensionality Check:** Enforces strict alignment with 1024-D vectors (`EMBEDDING_DIM = 1024, float32`).
 - **Real Vector Search Probe (`test_index_search_returns_results`):** Runs a dummy k -NN search ($K = 5$) directly on the FAISS index using vector `embeddings[0:1]` to confirm the search engine returns valid, non-negative L_2 distance metrics.
-

3. Defense Talking Points & Anticipated Questions

If your defense committee asks about your testing framework:

- “How do you test that your RAG pipeline doesn’t ingest stale or outdated events?” > “We enforce automated temporal unit tests (*TestDateFilter*). Instead of comparing against system runtime—which causes false test failures on older valid snapshots—we record the exact UTC fetch timestamp in `fetch_metadata.json` and assert that 100% of

ingested events fell within the maximum allowed age threshold (365 days) at the time of retrieval.”

- **“Why test vector database synchronization in a unit test suite?”**
> *“FAISS manages raw vector arrays decoupled from text attributes. If a vector array becomes offset from its metadata file by even one record, the chatbot will retrieve text for event A while searching the vector of event B. Our unit tests assert exact 1 : 1 positional equality (`faiss.ntotal == len(metadata)`) before the application can launch.”*
- **“How do you prevent garbage data from entering the embedding space?”** > *“We enforce pre-embedding assertions: every raw record must pass strict null checks on title, description, and date fields, and every text chunk must validate structural markers (`'Événement :'`) and URL presence before being sent to the Mistral API.”*

This script executes **Offline Evaluation & Benchmarking** using **RAGAS (Retrieval Augmented Generation Assessment)**.

It runs your production RAG pipeline (`src.rag.langchain_chain.ask`) against a diverse, human-annotated ground-truth dataset (30 test questions across 6 domain categories) to quantify performance using an **LLM-as-a-Judge** framework.

Here is a structured overview and defense breakdown for this script.

1. High-Level Architecture & Workflow

1. **Production Chain Import:** The script directly imports `ask()` from `src.rag.langchain_chain`. This guarantees that RAGAS measures the exact chain (prompt template, top-*k* FAISS search, chunk formatting, and Mistral LLM configuration) that real users interact with in the Streamlit UI, preventing silent evaluation-to-production drift.
 2. **Benchmark Generation (`run_rag_on_dataset`):** Loops over 30 test questions across specific domains, recording generated answers, source text chunks, and vector UUIDs while applying pacing delays (`CALL_SLEEP = 1.5`) to respect API rate limits.
 3. **Automated Metrics Calculation (`run_ragas`):** Leverages Mistral AI (`mistral-small-latest`) as the LLM judge alongside `MistralAIEmbeddings` to compute standardized RAG performance scores.
 4. **Artifact Persistence:** Exports structured evaluation logs to `test_dataset.csv` and summary metrics to `ragas_results.csv`.
-

2. Benchmark Test Set Design

The dataset contains 30 questions divided into 6 strategic evaluation buckets to challenge both retrieval and generation capabilities:

Category	Count	Purpose
specific_event	6	Tests precise entity lookup (e.g., concert names, dates, venues)
date_based	5	Evaluates temporal filtering across months/weekends
category_based	5	Tests broader semantic grouping (e.g., classical vs. religious)
venue_based	4	Validates spatial retrieval accuracy for specific Parisian churches
out_of_scope	5	Tests guardrails and safety (e.g., weather, restaurants)
no_match	5	Verifies fallback handling when data is absent (e.g., bullfighting, football)

3. The 4 RAGAS Metrics Explained

RAGAS evaluates your pipeline across the standard **RAG Triad** plus retrieval completeness:

User Query

Context Precision
Context Recall

Answer Relevancy
(Evaluates Output)

Retrieved Contexts

Generated Answer

Faithfulness
(Groundedness Check)

1. **Faithfulness (Generation Groundedness):**

- **What it measures:** Calculates the proportion of statements in the generated response that can be directly inferred from the retrieved context.
- **Why it matters:** Detects hallucinations. A score near 1.0 guarantees the model is not making up false dates or locations.

2. **Answer Relevancy (Generation Quality):**

- **What it measures:** Evaluates how directly the answer addresses the user’s prompt without adding irrelevant fluff.
- **Why it matters:** Ensures the chatbot gives concise, direct answers instead of diverging off-topic.

3. **Context Precision (Retriever Signal-to-Noise Ratio):**

- **What it measures:** Measures whether relevant chunks are ranked higher in the top- k retrieved list compared to irrelevant ones.
- **Why it matters:** Evaluates FAISS vector retrieval quality and chunking strategy. High precision means top chunks contain the necessary facts.

4. **Context Recall (Retriever Coverage):**

- **What it measures:** Compares the retrieved context against the human-annotated `reference_answer` to check if all necessary facts were successfully fetched.
- **Why it matters:** Identifies whether missing facts in the final answer are due to poor retrieval (k too small) or poor generation.

4. Defense Talking Points & Defense Strategy

If your evaluation or defense committee asks about your testing and validation methodologies:

- **“How do you evaluate your RAG system objectively without relying on manual checks?”** > *“We implemented an automated evaluation framework using **RAGAS** and **LLM-as-a-Judge** with **mistral-small-latest**. We benchmark our system on a 30-question golden dataset categorized by event queries, date/venue filters, and out-of-scope edge cases to quantitatively measure retrieval precision and generation faithfulness.”*

- “How do you know your chatbot doesn’t hallucinate facts?” > “We track RAGAS **Faithfulness**. This metric decomposes the generated answer into discrete claims and verifies each claim against the retrieved context chunks. Furthermore, 33% of our test dataset consists of out-of-scope or non-existent event queries (e.g., football matches, bullfighting) to verify that the model correctly answers ‘Je ne sais pas’ when facts are absent from the vector database.”
- “Why import ask() from langchain_chain.py instead of running a simple evaluation loop in this file?” > “To eliminate software drift between evaluation and production. If we wrote isolated retrieval logic inside `evaluate.py`, optimizing our test scores would not guarantee performance in the live Streamlit UI. Evaluating the exact production chain function ensures our metrics accurately reflect the end-user experience.”

This script establishes the **Canonical RAG Chain** (`langchain_chain.py`) using **LangChain Expression Language (LCEL)**, **FAISS**, and **Mistral AI**.

It serves as the **Single Source of Truth** across the entire codebase—imported directly by both the Streamlit UI (`chatbot.py`) and the offline evaluation harness (`evaluate.py`), strictly adhering to the **DRY (Don’t Repeat Yourself)** software design principle.

Here is a structured, defense-ready technical breakdown of this script.

1. System Architecture & LCEL Execution Flow

The core RAG pipeline is built using **LangChain Expression Language (LCEL)**, providing a clean, declarative pipeline:

User Question → Retriever → Context Formatter → Prompt Template → ChatMistralAI → StrO
 Top-*k* FAISS Search Flatten Docs Strict Guardrails LLM Generation

```
chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

Key Components:

1. `wrap_faiss_in_langchain()`: Converts the raw binary FAISS index (built by `03_index.sh`) and its corresponding `metadata.json` into a

LangChain FAISS vector store in memory without re-embedding the raw texts.

2. **build_chain() with Caching:** Implements a singleton module-level cache (`_cached_chain`). Re-executing queries in the Streamlit UI reuses the pre-loaded FAISS index and initialized network connections rather than reloading files from disk on every user turn.
3. **call_with_retry():** Wraps both API calls (vector embedding lookups and text generation) in an **exponential backoff retry loop** (2^n second backoff up to 5 attempts). This gracefully handles API rate limits (429 HTTP status codes) during heavy batch query runs.

2. Guardrail Engineering & Prompt Strategy

The system prompt strictly limits hallucination risks using **Closed-Domain Grounding**:

Tu es un assistant spécialisé dans les événements culturels à Paris.

...

Règles strictes :

- Réponds **UNIQUEMENT** en te basant sur les événements fournis dans le contexte.
 - Si l'information n'est pas dans le contexte, dis clairement que tu ne sais pas.
 - Ne génère **JAMAIS** d'informations inventées sur des événements.
- **Temperature Control:** Set to 0.1 (`LLM_TEMPERATURE = 0.1`) to ensure deterministic responses and prevent creative hallucination.
 - **Fall-back Handling:** Out-of-scope or unindexed queries return an explicit "I don't know" ("**Je ne sais pas**") response rather than generating speculative answers.

3. Defense Talking Points & Anticipated Questions

If your defense committee asks about your pipeline design and integration:

- **“Why wrap your raw FAISS index in LangChain instead of calling FAISS directly?”** > *“Wrapping our FAISS index in LangChain’s FAISS vector store abstraction allows us to leverage LangChain’s standardized `Retriever` interface. This allows seamless integration into LCEL chains (`retriever | format_docs`) while retaining complete control over our underlying vector index structure.”*
- **“How do you prevent code drift between what you evaluate and what you serve to users?”** > *“We enforce a single canonical entry point: `ask()` in `langchain_chain.py`. Both the Streamlit interactive dashboard (`chatbot.py`) and the automated evaluation harness (`evaluate.py`) call*

this exact same function. There is zero duplicated retrieval or generation logic in our repository.”

- **“How does your pipeline handle API rate limiting and service throttling on free/shared tiers?”** > *“We wrapped all Mistral API invocations in `call_with_retry()`, an exponential backoff decorator that captures 429 HTTP errors or capacity warnings. On rate limits, the system pauses for 2^n seconds (2s, 4s, 8s, 16s, 32s) before retrying, ensuring batch operations complete without crashing.”*